

Mini Project Report

On

Stock Price Trend Prediction and Recommendation System

A web-based stock trend forecasting and Buy/Sell advisory application

BACHELOR OF TECHNOLOGY

in

ROBOTICS AND ARTIFICIAL INTELLIGENCE

MANAV RACHNA UNIVERSITY

Declared as State Private University vide Haryana Act 26 of 2014

Submitted by:

Raghav Gaur
Roll No: 2K23CSUN06016
Batch: 2023 – 2027

Department:

Robotics and Artificial intelligence
Faridabad
2026

STUDENT DECLARATION

I, Raghav Gaur (2K23CSUN06016), hereby declare that the project titled "Stock Price Trend Prediction and Recommendation System" is a record of original work carried out by me as an independent, self-initiated mini project undertaken to apply and extend my learning in applied machine learning and full-stack deployment.

I further declare that this project has not been submitted previously, in part or in full, for the award of any other degree, diploma, or certification in this or any other institution. All sources of information used in this project, including open-source libraries and public financial data APIs, have been properly acknowledged and cited in this report.

Date: _____

Place: Faridabad

Raghav Gaur

Roll No: 2K23CSUN06016

Signature: _____

ABSTRACT

Retail investors routinely make buy-or-sell decisions without access to the quantitative tooling available to institutional traders. This project addresses that gap by building a lightweight, browser-accessible web application that forecasts short-term stock price direction and converts that forecast into a plain-language Buy/Sell recommendation for any publicly traded ticker.

The system is built on Flask and fetches live historical Open-High-Low-Close (OHLC) data for a user-supplied ticker (e.g., TSLA, AAPL, AMZN) using the yfinance API. Three engineered features — the previous day's closing price (Close_lag1), the price two days prior (Close_lag2), and a 10-day simple moving average (MA10) — are derived from the raw closing-price series. These features are used to train a scikit-learn Linear Regression model on a chronological 70/30 train-test split (shuffling disabled to preserve time order), which predicts the next closing price for each point in the held-out test window.

Model quality is reported using Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) computed on the test set. From the predicted price series, the system derives a day-by-day Buy/Sell/Hold signal by comparing consecutive predictions, an overall trend label (Upward/Downward), a headline recommendation (Consider Buying/Consider Selling), and a hypothetical expected return on a fixed ₹10,000 investment. Results are rendered through an interactive Plotly chart overlaying actual versus predicted prices, embedded in a dark-themed HTML interface built with Flask templates and inline CSS.

The complete application is deployed as a publicly accessible web service on Render, requiring no local installation, GPU, or specialised hardware. The project demonstrates a practical, end-to-end pipeline — from live data ingestion to model training to interactive visualisation and deployment — using only classical, interpretable machine learning, making it a reproducible baseline for further work in retail-facing financial forecasting tools.

Keywords: *Stock Price Prediction, Linear Regression, yfinance, Time-Series Forecasting, Flask, Plotly, Financial Machine Learning, Web Deployment*

Table of Contents

1. Introduction	7
1.1 Background	7
1.2 Problem Statement	7
1.3 Motivation	8
1.4 Scope	8
1.5 Objectives	8
1.6 Applications	9
1.7 Mapping with Relevant SDGs	9
2. Literature Review and Comparative Analysis	10
3. Proposed Solution	12
3.1 Methodology	12
3.2 System Design	12
3.3 Hardware / Software Requirements	13
3.4 Algorithm / Pseudocode	13
3.5 Dataset Description	14
3.6 Initial Prototype Demonstration	14
3.7 Achievement of Objectives as per Plan	15
4. Project Planning	16
4.1 Gantt Chart	16
4.2 Team Contribution	16
4.3 Work Breakdown Structure	17
5. Conclusion and Future Directions	18
6. References	20

1. Introduction

1.1 Background

Stock markets generate vast quantities of publicly available price and volume data every trading day, yet the majority of individual retail investors interpret this data manually or rely on subscription-based advisory platforms. Time-series forecasting using classical statistical and machine learning techniques offers a low-cost, transparent alternative: given a ticker's historical closing prices, simple regression-based models can approximate short-term price direction with reasonable accuracy and full interpretability of the underlying features.

Python's open-data ecosystem — particularly the `yfinance` library, which wraps Yahoo Finance's public endpoints — makes it possible to pull years of daily OHLC data for any listed ticker with a single function call, removing the traditional barrier of paid market-data subscriptions for prototype and educational systems.

1.2 Problem Statement

Retail investors lack an accessible, no-cost tool that converts raw historical price data into an actionable, day-level trend forecast and recommendation. Existing brokerage-integrated advisory tools are either paywalled, tied to a specific platform, or built as opaque black-box models that do not expose the underlying logic. There is a need for a lightweight, transparent, ticker-agnostic web application that any user can query for an instant trend read and a Buy/Sell signal.

1.3 Motivation

The motivation for this project came from the observation that most beginner-friendly market tools either oversimplify prediction into a static chart or require paid APIs and heavy infrastructure. This project set out to prove that even a classical, fully interpretable model — Linear Regression over lag and moving-average features — can be packaged into a usable, visually clean, publicly deployed web tool without GPU or deep learning infrastructure, while also serving as a practical exercise in applied ML and full-stack deployment for the author.

1.4 Scope

The current scope of the project encompasses the following:

- Prediction of next-step closing price for any valid, actively traded stock ticker supported by Yahoo Finance
- Feature engineering limited to lagged closing prices ($t-1$, $t-2$) and a 10-day moving average
- Model training using scikit-learn Linear Regression on a chronological train/test split
- Derivation of day-level Buy/Sell/Hold signals, an overall trend label, and a headline recommendation
- Interactive Plotly visualisation of actual vs. predicted price with hover-level return and signal detail
- Web-based deployment via Flask, hosted publicly on Render

Out of scope for the current version: intraday (sub-daily) prediction, multi-day-ahead forecasting, sentiment or news-based features, portfolio-level optimisation, and real-money trade execution.

1.5 Objectives

The primary objectives of this project are:

- To build an end-to-end pipeline that fetches live historical stock data for any user-specified ticker
- To engineer a minimal, interpretable feature set (lag prices + moving average) for next-step price regression
- To train and evaluate a Linear Regression model using RMSE and MAE on a time-ordered test split
- To translate model output into a human-readable trend label, Buy/Sell recommendation, and expected return figure
- To visualise actual vs. predicted price interactively using Plotly, embedded in a clean dark-themed UI
- To deploy the system as a publicly accessible, installation-free web application

1.6 Applications

- Quick-reference trend and sentiment check for retail investors before placing a manual trade
- Educational tool for teaching time-series regression and feature engineering concepts in ML courses
- Baseline benchmark for comparing more advanced forecasting models (LSTM, ARIMA, Prophet) in future work
- Portfolio-tracking dashboards that need a lightweight, embeddable trend widget per ticker
- Personal-finance learning apps aimed at first-time market participants

1.7 Mapping with Relevant SDGs

This project aligns with the following United Nations Sustainable Development Goals (SDGs):

- SDG 1 (No Poverty) & SDG 8 (Decent Work and Economic Growth): Improves financial literacy and decision-making tools available to individuals, supporting more informed wealth-building.
- SDG 9 (Industry, Innovation and Infrastructure): Demonstrates practical application of open-source machine learning and data infrastructure to a real-world financial use case.
- SDG 10 (Reduced Inequalities): Provides free, publicly accessible market-analysis tooling that reduces the gap between institutional and retail investors.
- SDG 17 (Partnerships for the Goals): Built entirely on open-source frameworks (scikit-learn, Flask, Plotly) and open financial data (Yahoo Finance via yfinance), maximising accessibility and reproducibility.

2. Literature Review and Comparative Analysis

Stock price forecasting has been studied extensively across statistics and machine learning. Classical econometric approaches such as ARIMA and GARCH model price series as autoregressive stochastic processes and remain widely used for their interpretability, though they struggle to capture non-linear market dynamics. Machine learning approaches — Support Vector Regression, Random Forests, and gradient boosting — improved on this by learning non-linear relationships between engineered features and future price, at the cost of reduced interpretability.

Deep learning architectures, particularly Long Short-Term Memory (LSTM) and GRU-based recurrent networks, have shown strong performance on sequential financial data by learning temporal dependencies directly from raw price windows rather than hand-engineered lag features. However, these models require substantially larger training datasets, GPU acceleration for practical training times, and are prone to overfitting on noisy, non-stationary financial series without careful regularisation.

Simple linear models regressed on lagged prices and moving averages remain a common and defensible baseline in this literature: they are fast to train on CPU, fully interpretable (each coefficient has a direct economic reading), and — over short forecast horizons where price series exhibit strong autocorrelation — often perform competitively with far more complex models. This project adopts that baseline deliberately, treating lag-1, lag-2, and a 10-day moving average as the minimal feature set sufficient to capture short-term momentum and trend, and prioritising a complete, deployed, interpretable pipeline over marginal model complexity.

A comparative summary of relevant approaches is presented below:

Approach	Input Features	Compute	Interpretability	Typical Use
ARIMA / GARCH	Raw price series	CPU	High	Short-term statistical forecasting
SVR / Random Forest	Technical indicators	CPU	Medium	Non-linear feature-based prediction
LSTM / GRU	Windowed raw sequences	GPU (practical)	Low	Long-horizon sequential learning
Linear Regression + Lags (This Project)	Close_lag1, Close_lag2, MA10	CPU	High	Fast, interpretable, deployable baseline

Table 2.1: Comparative analysis of stock forecasting approaches

The present work positions itself deliberately at the interpretable, low-compute end of this spectrum. Its purpose is not to beat state-of-the-art forecasting accuracy but to deliver a complete, transparent, and publicly usable pipeline, with the Future Directions (Section 5.2) explicitly outlining a planned upgrade path toward LSTM-based sequential modelling once the current baseline and deployment are stable.

3. Proposed Solution

3.1 Methodology

The system follows a modular, four-stage pipeline: (1) live data ingestion for a user-specified ticker, (2) feature engineering from the raw closing-price series, (3) model training and evaluation on a chronological split, and (4) signal derivation, visualisation, and rendering back to the user through the web interface. Each request is handled statelessly — data is re-fetched and the model is re-trained fresh on every query — keeping the system simple to reason about and free of stale-model risk.

On form submission, the Flask backend downloads daily OHLC data for the requested ticker between 2020-01-01 and 2024-01-01 via `yfinance`, engineers three features from the closing-price column, fits a Linear Regression model on a 70/30 chronological split, and generates predictions on the held-out test window. Predicted prices are converted into day-over-day Buy/Sell/Hold signals, an overall trend label, a headline recommendation, and an expected return on a hypothetical ₹10,000 investment, all of which are passed to the template alongside a Plotly chart of actual versus predicted price.

3.2 System Design

The system comprises the following key components:

- `app.py`: The Flask application entry point. Handles routing, form input (custom or dropdown-selected ticker), data download, feature engineering, model training, signal generation, and template rendering.
- `templates/index.html`: Renders the dark-themed UI — ticker input form, and (on submission) the prediction summary cards, recommendation banner, and embedded Plotly chart.
- `yfinance` integration: Fetches historical OHLC data directly from Yahoo Finance at request time, requiring no local database or pre-downloaded dataset.
- Plotly chart engine: Renders an interactive dual-line chart (actual vs. predicted price) with custom hover tooltips showing per-point return percentage and Buy/Sell/Hold signal.
- `Procfile / runtime.txt`: Deployment configuration for Render, defining the WSGI entry point and Python runtime version for the hosted environment.

3.3 Hardware / Software Requirements

Hardware Requirements:

- Any device with a modern web browser (desktop, laptop, tablet, or mobile)
- Server-side: standard cloud compute instance (no GPU required) — as provisioned by Render
- Internet connection for live data retrieval via `yfinance` and for accessing the hosted application

Software Requirements:

- Python 3.x
- Flask (web framework)
- `yfinance` (market data retrieval)
- `pandas`, `numpy` (data handling and numerical computation)
- `scikit-learn` (Linear Regression, `train_test_split`, evaluation metrics)
- Plotly (`plotly.graph_objs`, `plotly.io`) for interactive charting

- Unicorn / Render deployment stack (Procfile-driven)

3.4 Algorithm / Pseudocode

Algorithm 1: Stock Trend Prediction Pipeline

INPUT: Ticker symbol (string) OUTPUT: Predicted price, trend, recommendation, chart

```

1. Download OHLC data for ticker from 2020-01-01 to 2024-01-01 via yfinance
2. IF data is empty: return error "No data found for ticker"
3. Compute MA10 = 10-day rolling mean of Close
4. Compute Close_lag1 = Close shifted by 1 day; Close_lag2 = Close shifted by 2 days
5. Drop rows with NaN (initial rolling/lag window)
6. X = [Close_lag1, Close_lag2, MA10]; y = Close
7. Split X, y into train (70%) / test (30%) – shuffle = False (preserve time order)
8. Fit LinearRegression on (X_train, y_train)
9. predicted = model.predict(X_test)
10. rmse = sqrt(MSE(y_test, predicted)); mae = MAE(y_test, predicted)
11. FOR i in 1..len(predicted): signal[i] = "Buy" if predicted[i] > predicted[i-1] else "Sell"
12. trend = "Upward" if predicted[-1] > predicted[-2] else "Downward"
13. recommendation = "Consider Buying" if trend == "Upward" else "Consider Selling"
14. expected_return = (predicted[-1] - y_test[-1]) / y_test[-1] * 10000
15. Render Plotly chart of Actual vs. Predicted price with return/signal hover data
16. RETURN predicted price, trend, recommendation, rmse, mae, chart

```

3.5 Dataset Description

Unlike a static, pre-collected training set, this system draws its dataset dynamically at request time: daily OHLC (Open, High, Low, Close, Volume) records for the requested ticker, spanning 2020-01-01 to 2024-01-01 (approximately four years / ~1,000 trading days per ticker), retrieved live from Yahoo Finance via the yfinance API. No image, audio, or manually labelled data is involved — every feature is derived arithmetically from the Close price column, which keeps the pipeline free of labelling cost or storage overhead.

Feature	Description	Derivation
Close_lag1	Previous trading day's closing price	Close.shift(1)
Close_lag2	Closing price two trading days prior	Close.shift(2)
MA10	10-day simple moving average of Close	Close.rolling(window=10).mean()
Target (y)	Same-day closing price	Close

Table 3.1: Engineered feature set used for model training

3.6 Initial Prototype Demonstration

The initial prototype is fully functional and publicly deployed at <https://stock-proj-14.onrender.com/>.

The prototype demonstrates:

- A dark-themed landing page accepting either a custom ticker entry or a selection from popular tickers
- Live retrieval and processing of historical price data for the entered ticker on form submission
- A summary panel showing current price, predicted price, trend label, and Buy/Sell recommendation
- Reported RMSE and MAE for the fitted model on the held-out test window
- An interactive Plotly chart plotting actual vs. predicted price, with hover tooltips showing per-point return percentage and signal
- A calculated expected return figure on a hypothetical ₹10,000 investment

3.7 Achievement of Objectives as per Plan

Objective	Target	Status
Live data ingestion for any ticker	Functional, any valid Yahoo Finance ticker	Achieved
Feature engineering (lags + MA10)	Functional	Achieved
Linear Regression model + evaluation	RMSE / MAE reported	Achieved
Trend label + Buy/Sell recommendation	Functional	Achieved
Interactive Plotly visualisation	Functional	Achieved
Public web deployment	Publicly accessible URL	Achieved (Render)
Multi-day-ahead forecasting	Stretch goal	Not yet started

Table 3.2: Objective achievement status

4. Project Planning

4.1 Gantt Chart

The project was executed over a 4-week development cycle, organised into the following phases:

Phase	W1	W2	W3	W4
Requirement study & tech stack selection	■			
Data pipeline (yfinance integration)	■	■		
Feature engineering & model training		■	■	
Signal logic & recommendation engine			■	
Flask + Plotly UI development		■	■	
Testing & debugging			■	■
Deployment (Render) & report				■

Table 4.1: Project Gantt Chart (shaded cells indicate active development weeks)

4.2 Team Contribution

This project was independently conceived, developed, tested, and deployed by a single contributor. The table below breaks contribution down by functional area rather than by team member, for clarity.

Team Member	Responsibility	Contribution %
Raghav Gaur	End-to-end development: yfinance data pipeline, feature engineering, Linear Regression model, signal/recommendation logic, Flask backend, Plotly visualisation, UI, Render deployment, and report writing.	100%

4.3 Work Breakdown Structure

WBS Level 1: Stock Price Trend Prediction and Recommendation System

- WBS 1.1: Research and Planning
 - WBS 1.1.1: Study of forecasting approaches and technology stack selection
 - WBS 1.1.2: Feasibility analysis of yfinance for live data retrieval
- WBS 1.2: Data Engineering
 - WBS 1.2.1: yfinance integration for live OHLC data download
 - WBS 1.2.2: Feature engineering (Close_lag1, Close_lag2, MA10)
 - WBS 1.2.3: Data cleaning (NaN handling from rolling/lag windows)
- WBS 1.3: Model Development
 - WBS 1.3.1: Linear Regression implementation with scikit-learn
 - WBS 1.3.2: Chronological train/test split (70/30, shuffle disabled)
 - WBS 1.3.3: Model evaluation using RMSE and MAE

- WBS 1.4: Application Development
 - WBS 1.4.1: Signal derivation logic (Buy/Sell/Hold, trend, recommendation)
 - WBS 1.4.2: Expected-return calculation on hypothetical investment
 - WBS 1.4.3: Plotly interactive chart integration
 - WBS 1.4.4: Flask routing and dark-themed HTML/CSS interface
- WBS 1.5: Testing and Deployment
 - WBS 1.5.1: Local testing across multiple tickers and edge cases (invalid ticker, empty data)
 - WBS 1.5.2: Render deployment (Procfile, runtime.txt configuration)

5. Conclusion and Future Directions

5.1 Conclusion

This project successfully demonstrates an end-to-end, publicly deployed stock trend prediction and recommendation system built entirely on classical, interpretable machine learning. By combining live data retrieval via yfinance with a minimal, well-understood feature set (lagged closing prices and a 10-day moving average) and a Linear Regression model, the system produces a usable next-step price forecast, trend label, and Buy/Sell recommendation for any valid ticker, evaluated transparently through RMSE and MAE.

The Flask + Plotly architecture keeps the system lightweight, dependency-light, and free of GPU requirements, while the Render deployment makes it publicly accessible without local installation. The project validates that a simple, fully interpretable pipeline is sufficient to deliver a complete, functioning financial forecasting tool, and provides a stable foundation for the more advanced modelling directions outlined below.

5.2 Future Directions

- Replace Linear Regression with an LSTM or GRU-based sequential model trained on rolling windows of raw price/volume data to capture non-linear temporal dependencies
- Extend the feature set with technical indicators such as RSI, MACD, and Bollinger Bands to improve signal quality
- Support multi-day-ahead forecasting instead of single-step next-day prediction
- Incorporate sentiment features from financial news or social media as auxiliary predictors
- Add backtesting functionality to simulate the recommendation engine's returns over historical periods
- Introduce user accounts with saved watchlists and email/SMS alerts on trend changes
- Cache/store fetched OHLC data to reduce repeated calls to yfinance and improve response latency
- Add confidence intervals or prediction-uncertainty bands to the Plotly chart
- Containerise the application with Docker for reproducible deployment across environments

What else can be added:

- Portfolio mode — track predicted vs. actual performance across multiple tickers simultaneously
- Comparative model view — show Linear Regression, ARIMA, and LSTM predictions side-by-side for the same ticker
- Indian market mode — extend ticker support and currency formatting for NSE/BSE-listed stocks specifically

6. References

- [1] Rinaldo, A. (2020). Time-series regression and lag-feature engineering for financial forecasting. *Journal of Financial Econometrics*.
- [2] yfinance Documentation. (2023). Yahoo Finance market data downloader. <https://pypi.org/project/yfinance/>
- [3] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [4] Plotly Technologies Inc. (2023). Plotly Python Open Source Graphing Library. <https://plotly.com/python/>
- [5] Flask Documentation. (2023). Pallets Projects. <https://flask.palletsprojects.com/>
- [6] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array Programming with NumPy. *Nature*, 585, 357–362.
- [7] McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- [8] Fischer, T., & Krauss, C. (2018). Deep learning with long short-term memory networks for financial market predictions. *European Journal of Operational Research*, 270(2), 654–669.
- [9] Ariyo, A. A., Adewumi, A. O., & Ayo, C. K. (2014). Stock price prediction using the ARIMA model. *UKSim-AMSS 16th International Conference on Computer Modelling and Simulation*.
- [10] Render. (2023). Render Cloud Application Hosting. <https://render.com/docs>
- [11] Investopedia. (2023). Moving Average (MA) Definition. <https://www.investopedia.com/terms/m/movingaverage.asp>